ZhuLong: An Execution-Grounded LLM Agent for EDA Scripting

Anonymous Author(s)

Abstract

EDA scripting with tool-specific, often undocumented APIs remains a long-tail bottleneck that existing LLM-based tooling fails to address. This paper presents **ZhuLong**, an execution-grounded LLM coding agent for PyAether, SKILL, and Tcl that combines API retrieval, documentation inspection, and sandbox execution via unified MCP tools into a closed generate–execute–refine loop. We further provide a **theoretical grounding** for why runtime observation is necessary: EDA correctness is a *situated* property that cannot be guaranteed from static knowledge, and we formalize this "grounding gap" with a Bayesian argument showing it is bounded by observation fidelity—not model scale. The Harness supplies precisely that observation, by reading the environment's documented API facts and by reading back what executing code does to the live design.

We evaluate ZhuLong on **EDA-Eval-PyAether**, a benchmark of 158 real-world tasks with assertion-based execution, where the complete system achieves **82.1%** Pass@1 (±3.0, 5-run mean) in the commercial Empyrean Aether environment, substantially outperforming a pure LLM baseline (**10.5%**). Ablation studies show the execution loop is decisive: adding sandbox execution to retrieval yields +10.3 pp, while removing either execution (−10.3 pp) or retrieval (−1.1 pp) reduces accuracy—consistent with the theoretical prediction. The main quantitative results are measured on the 158-task PyAether slice; a cross-language reproduction of the component ablation on the SKILL (158) and Tcl (133) slices of the unified **EDA-Scripting-Bench** (449 tasks total) reproduces the same structure—the complete system is the top and the prior-only baseline the bottom on every language—and the remaining ablation axes are being re-measured on the full benchmark.

CCS Concepts

• **Hardware** → **Software tools for EDA**.

Keywords

LLM Agents for EDA, EDA Scripting, Code Generation, Sandbox Execution, Execution Grounding

ACM Reference Format:
Anonymous Author(s). 2026. ZhuLong: An Execution-Grounded LLM Agent for EDA Scripting. In *Proceedings of* . ACM, New York, NY, USA, 12 pages.

1 Introduction

Large language models (LLMs) have demonstrated strong capabilities in general-purpose code generation [17, 23, 25]. However, applying LLMs to EDA scripting—where engineers routinely manipulate in-memory design databases, create layout instances, route schematics, and query netlists—remains challenging. First, EDA APIs are long-tailed and tool-specific; many rarely appear in public training corpora. Second, correctness cannot be verified statically: it depends on observable side effects in a real EDA environment, such as modified layouts or updated schematics. Third, without execution feedback, static generation cannot diagnose errors or iteratively refine incorrect code [3, 5]. Existing LLM-based EDA work has mainly focused on HDL generation [15, 21], while scripting with real tool APIs—especially with incomplete documentation and sandbox execution—remains underexplored.

This paper presents **ZhuLong**, an execution-grounded LLM coding agent for PyAether, SKILL, and Tcl. Named after a mythical creature whose opened eyes bring light, ZhuLong aims to illuminate opaque EDA APIs and automate tool-specific scripting workflows. Built on Cline-CLI [7], ZhuLong extends three MCP tools [16]—`search_apis`, `get_api_details`, and `run_code`—that form a closed loop: retrieve candidate APIs, inspect their documentation, execute code in the EDA environment with execution feedback, and iteratively refine based on observed errors or outputs. All tools accept a unified `lang` parameter, so the same agent logic covers PyAether, SKILL, and Tcl. Beyond the engineering, we provide a **theoretical foundation** (Section 4): EDA scripting correctness is a *situated* property that cannot be guaranteed from static knowledge, and a Bayesian analysis shows the resulting "grounding gap" is bounded by observation fidelity—not model scale—explaining why runtime observation is essential, and why the Harness must supply both halves of it: reading the environment's documented API facts, and reading back what executing code does to the live design.

The main contributions are:

- **An account of *which* factor sets the ceiling, with measurements that distinguish them (RQ1).** We formalize the *grounding gap*: EDA correctness is a situated (L4) property that no offline knowledge can guarantee, and a Bayesian argument shows the resulting gap is bounded by *observation fidelity* rather than by the capacity of the model (Section 4). We answer RQ1 from both sides—laddering the observation channel with the LLM fixed, and changing the prior with the harness fixed—and the framework is stated *before* the measurements, so that its claims are predictions the experiments could refute rather than explanations fitted afterwards.
- **A harness whose necessary components we isolate (RQ2).** ZhuLong couples three unified MCP tools—`search_apis`, `get_api_details`, and `run_code`—so that API retrieval and documentation inspection

,

(the *observation channel*), sandbox execution (the *effector*), and the generate–execute–refine loop (the *belief update*) form one closed loop, driven by a single `lang` parameter so the same agent logic covers PyAether, SKILL, and Tcl (Section 3). In a commercial Empyrean Aether environment, ZhuLong attains **82.1%** Pass@1 on the 158-task PyAether slice, versus **10.5%** for a pure LLM and **71.8%** for a retrieval-augmented static baseline. The same component ordering reproduces across the SKILL and Tcl slices of EDA-Scripting-Bench (Section 6.2.4). Ablation shows that the three components are *jointly necessary and not interchangeable*: adding sandbox execution to RAG yields **+10.3 pp**, while removing execution (−10.3 pp) or retrieval (−1.1 pp) reduces accuracy, and bounding the belief update degrades it monotonically (Section 6).

- **A cross-language benchmark on which the same structure can be re-measured per toolchain (RQ3). EDA-Scripting-Bench** places PyAether, SKILL and Tcl under one task schema, one assertion criterion and one harness, so that the structure of RQ1–RQ2 can be re-measured slice by slice with nothing changed but the toolchain (Section 5). Its completed PyAether slice, **EDA-Eval-PyAether** (158 tasks from four sources, expert-reviewed), serves as the first standardized PyAether benchmark; the SKILL and Tcl slices complete the suite for reproducible comparison of LLM-based scripting agents. What we release is the **harness, the task schema and the evaluation protocol**; the task cases themselves derive from vendor documentation and customer designs and cannot be redistributed.

2 Background and Motivation

We target two widely-used EDA scripting interfaces: PyAether for Empyrean Aether [9] and SKILL for Cadence Virtuoso [20]. Despite their prevalence, scripting for both environments poses challenges beyond what generic LLM code generators can address, and the community lacks basic infrastructure to measure progress.

EDA scripting is fundamentally different. Prior work on LLM code generation has succeeded in domains where correctness is verified by unit tests [2, 6]. EDA scripting presents three distinct challenges: (i) APIs are long-tailed, tool-specific, and poorly documented; (ii) correctness depends on side effects in a live, stateful database, not return values; and (iii) no benchmark exists to evaluate whether generated scripts actually work.

Prior work stops short of execution. Recent efforts like LayoutCopilot [14], ChaTCL [19], and RAG-EDA [18] combine LLMs with static knowledge bases. Yet they all share a critical limitation: they do not execute generated code. Correctness is assessed by human judgment [14, 19] or retrieval quality [18], not by whether the script actually modifies a layout. Consequently, errors go undiagnosed, and the gap between "looks plausible" and "actually works" remains unbridged.

The root problem: no benchmark, no progress. Without a standardized benchmark, the community cannot systematically compare approaches or measure progress—each prior work uses ad-hoc tasks with different protocols. Recent concurrent benchmarks for Tcl/Innovus flows [13, 24] confirm the growing recognition of this gap, yet target different toolchains. A dedicated PyAether/SKILL benchmark remains absent.

Our approach. ZhuLong addresses the technical challenges through sandbox execution feedback—closing the loop from generation to execution to refinement—together with a **theoretical grounding** (Section 4) that explains *why* execution is necessary. Crucially, we introduce **EDA-Eval-PyAether**, the first standardized benchmark for PyAether scripting. Together, these establish the methodology, the theory, and the measurement infrastructure for advancing LLM-based EDA scripting. The contribution boundary of this paper is the *harness*—the three MCP tools (`search_apis`, `get_api_details`, `run_code`), the evaluation protocol, and the benchmark; the *agent backbone* (the underlying model and its training) is the subject of a companion work, BaiZe [**?**], which we cross-reference without re-reporting its results.

3 System Design

3.1 Overall Architecture

ZhuLong is built on Cline-CLI [7], an open-source framework for autonomous coding agents, extended with EDA-specific retrieval, documentation, and execution capabilities. Throughout, we call the resulting runtime an **ICL-type autonomous agent**: an agent that decides its own next action, keeps no explicit posterior, and maintains its belief state implicitly by in-context learning, as formalized in Section 4. We choose Cline-CLI for its mature permission controls, human-in-the-loop support, and enterprise readiness. As shown in Figure 1, the system comprises three components: the LLM-based agent runtime, the API knowledge base, and the EDA execution environment. The agent interacts with these components through three MCP tools.

3.2 Three MCP Tools

ZhuLong extends three MCP tools [1], each accepting a unified `lang` parameter (`pyaether`, `skill`, or `tcl`) to support all three platforms with identical agent logic:

- `search_apis(query, lang)` retrieves relevant APIs via semantic and keyword-based search. We index APIs by name and description using BGE-M3 embeddings [4] with FAISS [8] for efficient similarity search, returning the top-K results ($K = 5$ by default) with confidence scores and descriptions.
- `get_api_details(api_name, lang)` fetches comprehensive documentation for a specified API, including parameter types, return values, and examples.
- `run_code(code, mode, lang)` executes the generated code in the EDA environment. It supports two modes:

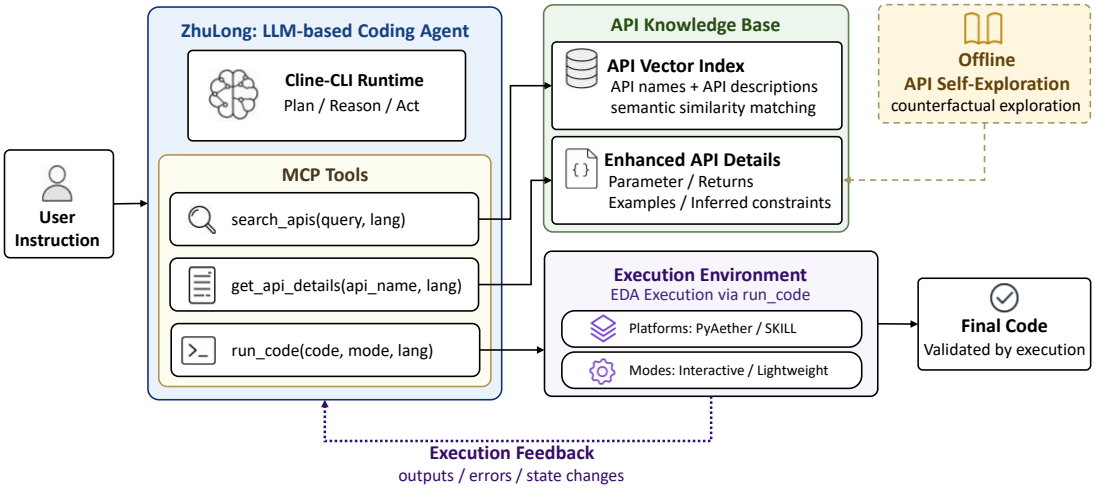


Figure 1: ZhuLong system architecture. The agent interacts with API retrieval, documentation, and EDA execution via three unified MCP tools. Execution feedback supports iterative refinement.

(1) **lightweight**—runs in a sandboxed environment without a GUI session (standalone PythonE for PyAether, or SKILL in batch mode for Virtuoso), capturing outputs, errors, and state changes for iterative refinement; (2) **interactive**—runs inside an open EDA session via the agent-EDA bridge (socket communication), enabling operations on unsaved layouts and schematics.

3.3 Agent Workflow

The agent iteratively refines its output through execution feedback [7]. A typical trajectory begins with reasoning over the user instruction, followed by API retrieval via `search_apis` when needed. Candidate APIs are then inspected via `get_api_details`, and code is generated accordingly. The agent executes the code via `run_code`, observes the result, and re-plans—revising the code, retrieving alternative APIs, or terminating. This loop continues until success or the agent determines further attempts are unlikely to succeed. The agent is not constrained to a fixed sequence; it may skip retrieval when it already knows the required API, or conduct multiple rounds of retrieval when initial results lack confidence.

4 Theoretical Grounding: The Grounding Gap

Our empirical results show that the execution loop is decisive—adding sandbox execution to retrieval yields +10.3 pp (McNemar $p<10^{-8}$), while removing either execution (−10.3 pp, $p<10^{-8}$) or retrieval (−1.1 pp, $p=0.61$, not significant) reduces accuracy. The retrieval-removal gap is not statistically significant on the PyAether-only slice, which actually strengthens the "execution > retrieval" narrative: the effector, not the documentation channel, is what the correctness criterion depends on. This section explains *why* this is not an accident of our implementation but a structural property of EDA scripting. We condense the *grounding gap* framework: EDA scripting correctness is a *situated* property that cannot be guaranteed from offline knowledge, and execution is the only channel that observes the current design state. The gap is bounded by observation fidelity, not model scale.

4.1 A Four-Level Hierarchy of Information

To make precise what can be learned offline versus what must be observed online, we distinguish four levels of information (Table 1). The distinction applies to any agent whose knowledge is fixed by offline learning before deployment:

- **L1–Syntactic Correctness**: well-formedness of symbols according to formal rules. Captured by formal grammars and the syntactic capacities of modern LLMs.
- **L2–Physical Plausibility**: what is physically plausible or generally expected—objects do not pass through walls, water flows downhill. L2 captures robust statistical patterns of the world, not logical certainties; these regularities are general and acquirable offline, constituting the agent's *prior* over how the world typically behaves.
- **L3–Historical Factuality**: what has actually occurred or is documented as fact. L3 concerns actual rather than merely possible states of affairs. Like L1 and L2 it can be acquired offline, but it is *past-oriented*: it describes what was true as of the training cutoff, with no connection to what is true *now*.
- **L4–Situated Truthfulness**: what is true *right now*, in this specific environment. Three characteristics define it: *indexicality*—truth depends on context and changes

over time; *non-guaranteeability from offline data*—no offline dataset can *guarantee* the current state, though it may induce strong priors; and *real-time commitment*—L4 must be acquired continuously and acted upon promptly.

Two clarifications matter. First, L4 is not a semantic category of propositions; it is a *grounding requirement* imposed by indexical reference—the same proposition may be L3 at one moment and L4 at another, depending on whether it refers to a past fact or a current state. Second, the four levels are not a taxonomy of all knowledge: they separate what can be *known in advance* from what must be *observed now*. L4 is whatever must be *read* from the environment rather than *recalled* from memory.

For EDA scripting, the situated state is the live design database: which cell and view are selected, and what the layout or schematic contains at this moment. Situated states are not limited to the physical world—they include digital and informational states generally, and the user's current intent—so a clarification obtained from the user is an L4 observation as well.

The correctness criterion we adopt—that generated code executes without error and all assertions over the mutated design pass—is an L4 property. A script that is well-formed (L1), plausible (L2), and invokes documented functions (L3) can nonetheless violate an undocumented binding contract, depend on an internal selection state, or target the wrong cell, and thus fail the test oracle. No amount of offline knowledge can resolve such failures; they must be *observed* by acting on the live design and reading back the result. Four architectural consequences follow from L4's defining characteristics: a system trained offline and deployed frozen has no observation modules, no persistent situated state, no execution interface, and no real-time feedback loop. Every failure we analyze in Section 6.3 traces to one or more of these.

4.2 A Bayesian Formalization of the Grounding Gap

We cast grounding as decision-making under partial observability, mirroring POMDPs and Bayesian filtering [11, 22]; the formulation below is diagnostic rather than algorithmic—it is not a blueprint for tractable inference in high-dimensional state spaces, but a normative standard that makes precise *what* is missing: the observation likelihood Ω and the recursive update Φ.

Let Θ denote the space of L4 states (the live EDA design database), o_t a runtime observation, a_t a generated code action, and $\mathcal{K}$ the offline knowledge encoded in the LLM's frozen parameters (L1–L3). The agent maintains a belief over the current L4 state:

$$b_t(\theta) \triangleq p(\theta_t \mid o_{1:t},\, a_{1:t-1},\, \mathcal{K}), \tag{1}$$

updated recursively by Bayes' rule:

$$b_t(\theta) \propto p(o_t \mid \theta_t) \int_{\Theta} p(\theta_t \mid \theta_{t-1}, a_{t-1})\, b_{t-1}(\theta_{t-1})\, d\theta_{t-1}, \tag{2}$$

where $p(o_t \mid \theta_t)$ is the *observation likelihood* Ω, implemented by the Harness's observation modules (Section 4.3), and $p(\theta_t \mid \theta_{t-1}, a_{t-1})$ is the *transition model* $\mathcal{T}$ supplied by the LLM's prior over how EDA operations transform the design. Actions are chosen by minimizing expected cost:

$$a_t^* = \arg\min_{a \in \mathcal{A}} \mathbb{E}_{\theta_t \sim b_t}\big[\mathcal{C}(\theta_t, a, G) + \gamma\, V(b_{t+1})\big], \tag{3}$$

where $\mathcal{C}$ encodes the task assertions and γ a discount factor.

The *grounding gap* is the divergence between the posterior achievable with runtime observations and the posterior from the offline prior alone:

$$D_{\text{grounding}}(t) \triangleq D_{\text{KL}}(p(\cdot \mid o_{1:t},\, \mathcal{K},\, \text{env}) \,\|\, p_{\mathcal{K}}(\cdot \mid o_{1:t}))\,. \tag{4}$$

This divergence is bounded by the fidelity of the observation channel, not by the capacity of the model. If $I(\Theta_t; O_t \mid \mathcal{K}) = 0$—that is, the observation channel is silent about the state—no improvement in the prior can recover the current state. Scaling $\mathcal{K}$ improves L1–L3, but it cannot reduce the gap below the floor imposed by Ω.

The source of the gap is causal, not statistical. Parameters are fixed before deployment; facts that come into existence after the training cutoff, or states that depend on unrepeatable circumstances, cannot be present in them by definition. The environment at time t is not a function of the training corpus, and offline data and scaling affect only the prior, never the availability of runtime observation.

This separates grounding from learning-based adaptation. Fine-tuning and meta-learning are capabilities acquired during *training*: they improve the *prior* over L1–L3 distributions. Grounding occurs during *deployment* and requires the likelihood and a posterior update from observing o_t. A better prior does not eliminate the need for observation; the two are complementary—one offline, one online; one about knowledge, one about belief.

4.3 ZhuLong as an Instance of the Agent Harness

The formulation maps onto ZhuLong's architecture. The frozen LLM supplies the prior $\mathcal{K}$ (L1–L3), the transition model $\mathcal{T}$, and an approximation to the policy Π. The Harness supplies what the LLM lacks—observation, belief update, and execution—and ZhuLong's three MCP tools instantiate it: `search_apis` and `get_api_details` are the *observation modules* Ω, reading documented API facts out of the environment; `run_code` is the *effector* that acts on the live design database, whose resulting state is transduced into textual observations (stdout, errors, assertion outcomes). The replanning loop (Section 3.3) implements the belief update Φ: each observation o_t revises the agent's belief about which API contracts and compositions actually hold. The closed loop $b_t \to a_t \to o_{t+1} \to b_{t+1}$ is what turns one-shot generation into grounded search; grounding is a property of the coupled system, not of the model alone.

Table 1: The four levels of information

Level	Name	Core Question	Learnable Offline?
L1	Syntactic Correctness	Is this string well-formed?	Yes
L2	Physical Plausibility	Could this event happen?	Yes
L3	Historical Factuality	Did this event happen?	Yes
L4	Situated Truthfulness	Is this true *right now*?	**No**

4.4 Design Principles

The formalization yields three architectural principles: *freeze* $\mathcal{K}$, *maximize* the fidelity of Ω, and *run* Φ *continuously*.

Decouple knowledge from belief. Knowledge $\mathcal{K}$ is the static L1–L3 prior encoded in the LLM's parameters—general and context-free; belief $b_t(\theta)$ is the dynamic posterior maintained by the Harness—specific and context-dependent. Updating $\mathcal{K}$ with the limited observations available during deployment is inefficient and risks catastrophic forgetting, and even a prior enriched offline cannot substitute for observation; freezing $\mathcal{K}$ and absorbing adaptation in b_t yields stability and flexibility at once.

Optimize observation over the prior. The gap is bounded by the fidelity of the likelihood Ω, not by the expressiveness of the prior: $D_{\text{grounding}}(t)$ is bounded by $\|\tilde{p} - p_{\text{real}}\|$, a bound independent of model scale. Investment in observation—retrieval and documentation quality, execution read-back, state estimation—therefore has a direct and bounded effect, whereas investment in model scale has diminishing returns once the prior is sufficiently rich, because it cannot push the gap below the floor imposed by observation.

Design for continuous belief updating. Φ operates online: every observation yields a recursive update $b_t \mapsto b_{t+1}$. Two implications follow. First, Φ's update rate must exceed the timescale of L4 dynamics, otherwise b_t goes stale. Second, b_t must be a structured representation that captures uncertainty and history, so that the agent can recover from unexpected changes. In the ICL-type autonomous agent of Section 3, Φ has no explicit posterior: it is approximated by in-context learning, since each returned observation is appended and conditions the next decision, while the agent decides for itself when the result is good enough to submit. Because such an agent is autonomous, the quantity to bound is not a count of "rounds"—it has none—but how much evidence its belief state may ever accumulate: a graded budget on the executions it may observe, or a read-back that lags the action it reports.

These principles *account for* our ablation findings rather than merely restating them. Removing the observation channel ("ZhuLong w/o Sandbox") leaves b_t uninformative: with no Ω, L4 violations such as hidden type contracts and internal selection state (Section 6.3) are unobservable by construction, and Pass@1 drops by 10.3 pp. A retrieval-only system (RAG, 71.8%) improves the L1–L3 prior but still cannot observe L4, so its ceiling is bounded by what offline knowledge can guarantee.

Three diagnostic questions follow for any candidate system: (1) does it implement Ω with sufficient fidelity to track L4 dynamics? (2) does it maintain b_t as a structured state, updated through Φ after each observation? (3) does it approximate Π from offline knowledge and execute a_t through a causal interface? Section 6 answers them for ZhuLong, on the benchmark constructed in Section 5.

These principles and questions organize the evaluation into three research questions (Section 6.1): whether the ceiling is set by observation or by the prior (RQ1), which components are necessary and whether they are interchangeable (RQ2), and whether the structure follows from the domain class rather than from one toolchain (RQ3).

5 Benchmark Construction

5.1 Dataset: EDA-Scripting-Bench

For DAC, we assemble **EDA-Scripting-Bench**, a unified cross-language benchmark spanning three EDA scripting languages—PyAether, SKILL, and Tcl—under one task schema and one execution-based protocol. Every task carries a `lang` field and language-appropriate assertions, so a single agent harness (Section 3.3) can be scored uniformly across platforms. EDA-Scripting-Bench currently comprises three completed slices: **EDA-Eval-PyAether** (158 PyAether tasks), a 158-task SKILL slice, and a 133-task Tcl slice (spanning the Custom WaveView and Innovus runtimes), for **449 tasks in total**. The main quantitative results of this paper are measured on the PyAether slice; a cross-language reproduction of the component ablation on the SKILL and Tcl slices is reported in Section 6.2.4, and the remaining ablation axes are being re-measured on the full benchmark.

5.2 Dataset: EDA-Eval-PyAether (PyAether slice)

To evaluate LLM-based code generation for PyAether, we constructed **EDA-Eval-PyAether**, the 158-task PyAether slice of EDA-Scripting-Bench. Tasks were collected from four sources: API references (61.4%), documentation examples (27.2%), internal training materials (7.6%), and anonymized CAD practical cases (3.8%). Each task underwent manual review by EDA domain experts.

Table 2 shows the distribution of tasks across different EDA scenarios. Layout and Schematic operations constitute the majority (70.2%), reflecting the core focus of PyAether usage in physical design workflows.

Table 2: Task distribution by scenario

Scenario	Tasks	Percentage
Layout	64	40.5%
Schematic	47	29.7%
Design Management	24	15.2%
Others	23	14.6%
Total	158	100%

5.3 Evaluation Protocol

Each task consists of a natural language prompt, a function signature, and assertion-based test code. A task is considered passed if the generated code executes in the PyAether sandbox without errors and all assertions pass. The primary metric is Pass@1 [6]:

$$\text{Pass@1} = \frac{\#\text{ tasks with all assertions passed}}{\text{Total tasks}} \times 100\%$$

6 Experiments

6.1 Experimental Setup

6.1.1 Research Questions. The framework of Section 4 makes three testable claims: that the grounding gap is bounded by observation fidelity rather than by the prior; that bridging it requires a runtime architecture supplying observation, belief update and execution; and that this follows from the domain's structure rather than from one toolchain. We ask one research question per claim.

- **RQ1 (Ceiling: observation or prior?).** Is the achievable accuracy bounded by the fidelity of the observation channel Ω, or by the strength of the frozen prior $\mathcal{K}$? We answer from both sides: the observation-fidelity ladders (S1) vary Ω with the LLM fixed, and the prior-axis contrast (S3) varies $\mathcal{K}$ with the harness fixed.
- **RQ2 (Which components are necessary, and are they interchangeable?).** Does bridging the gap require observation, execution through a causal interface, and continuous belief update—and can one of the three substitute for another? S2 varies the effector's presence and the belief-update budget.
- **RQ3 (Domain class or toolchain?).** Does the same structure reproduce on a structurally different toolchain? The protocol of S1–S2 is re-run on the SKILL and Tcl slices of **EDA-Scripting-Bench**, so the comparison is made *within* one benchmark rather than across separate artifacts.

RQ1 and RQ2 test the framework's structural claims; RQ3 tests whether they follow from the domain's structure rather than from one toolchain. Because RQ1's two arms sit in different studies, we read them against each other rather than in isolation.

6.1.2 Benchmark and Evaluation Protocol. We evaluate ZhuLong on **EDA-Eval-PyAether**, a benchmark of 158 real-world PyAether scripting tasks. Following the evaluation protocol defined in Section 5, we adopt **Pass@1** (execution success rate) as the primary metric. Each task is evaluated in a single trace: the agent may iterate *within* that trace through the belief update Φ (Section 3.3), but a task is never restarted across traces; the timeout is set to 2500 seconds per execution. Every tool call is additionally mediated by a file-based `PreToolUse` hook—a JSON-in/JSON-out script dispatched by Cline's hook engine from the configured hooks directory—that enforces a tool whitelist, blocks shell and code-search tools, and confines all file access to a fresh, empty working directory; on each call it allows the call, rewrites it to a harmless no-op (`overrideInput`), or rejects it with an error message (`cancel`), with MCP tool calls passing through, so neither a task's own assertions nor any project copy of a solution is reachable from the workspace. The hook is identical in every configuration reported here and is thus part of the fixed protocol rather than a variable; it is also what makes the baselines interpretable. Traces terminated by the hook are identified separately so that they are not read as capability failures. To assess statistical reliability given the modest benchmark size (158 tasks), each configuration is evaluated over five independent runs with reshuffled task orderings and fresh agent contexts, and accuracy is reported as mean ± standard deviation over those runs. Quantities aggregated over traces rather than over runs—read-back counts, activation ratios and tool-call totals—are given as single aggregates over the pooled traces, without run-level dispersion; effect sizes (Δ) are computed from the mean values in the same table. Generation-stage failures—tasks that produce no executable code, e.g. due to timeout or an exhausted message budget—are counted as not-passed and remain in the 158-task denominator; for the S2 low-budget arms this generation failure (gen-fail) is the direct consequence of the Φ execution budget being exhausted, not an infrastructure fault. Run validity is gated separately: a run that suffers an infrastructure fault (license exhaustion, sandbox port unreachable, or insufficient disk) is discarded and re-run, and is not counted toward the five runs; the anti-leak hook is held identical across all arms. To isolate infrastructure degradation from ablation effects, the full-system anchor is required to reach at least 75% Pass@1 as a health floor—a run below it signals an infrastructure problem rather than a capability result and is re-run—while ablation arms have no such floor, so that their (sometimes low) scores reflect the true ablation effect. The dominant sandbox-execution effect (+10.3 pp) exceeds the spread by an order of magnitude, consistent with the theoretical account in Section 4. All figures in this section are provisional, obtained on the PyAether-only slice, and will be regenerated on the full EDA-Scripting-Bench (Section 5).

6.1.3 Baselines and ZhuLong Variants. To isolate the contribution of each component, we establish two baselines and two ZhuLong variants, as shown in Table 3. Read against the framework of Section 4, the table's columns name the

components a harness supplies: retrieval and documentation are the observation channel Ω, the sandbox is the effector through which an action reaches the live design. ZhuLong variants default to vector retrieval using a name+description index; S1 varies the fidelity of that index explicitly, and S2 adds a bound on the belief update Φ, which this table does not vary.

These columns name components that are realized at the harness boundary rather than inside the agent. The presence of each observable capability is switched by a tool-registration toggle—a single disabled-list in the server environment spanning the four core tools `search_apis`, `search_apis_by_keyword`, `get_api_details` and `run_code`: a disabled tool simply disappears from the agent's tool set, while every enabled tool keeps a byte-identical implementation across all arms. The sandbox executor is removed by unregistering `run_code`; retrieval is removed by unregistering the three retrieval tools; the anti-leak `PreToolUse` hook stays loaded in every arm.

Table 3: Configurations and their components

Configuration	Retrieval	Sandbox
Baseline 1 (Pure LLM)	✗	✗
Baseline 2 (RAG)	✓	✗
ZhuLong w/o Retrieval	✗	✓
ZhuLong (Full)	✓	✓

6.1.4 LLM Backbones. All main experiments use DeepSeek-V4-Pro as the primary backbone. To understand the impact of different LLMs, we additionally evaluate GLM-5.2, DeepSeek-V4-Flash, Kimi-K2.6, and Doubao-Seed-2.0-Pro.

6.2 Results and Ablations

The framework of Section 4 fixes what each study must vary. Table 4 gives the progressive build-up and the component ablation as an overview; the studies below then change one component at a time, with the LLM frozen unless stated otherwise.

6.2.1 S1: Observation fidelity (RQ1). Principle 2 predicts that with the model fixed, accuracy is bounded by the fidelity of the observation channel Ω. We vary Ω along the two axes through which it receives information, holding the LLM fixed in both.

Documentation axis. Table 5 holds the agent loop, the effector and the LLM fixed, and changes only how faithfully the environment's API facts are exposed: no observation at all; a low-fidelity index built from API names alone; a high-fidelity index that adds descriptions, BM25 term matching, abbreviation expansion and synonym redirect.

The documentation axis yields a non-monotone ordering: the low-fidelity rung (L) at $82.8 \pm 1.0\%$ exceeds the high-fidelity rung (H) at $71.8 \pm 2.5\%$. This is not an artifact of the index—it reflects the presence of the effector. Rung (L) couples a name-only index with the full execution loop

Table 4: Ablation study results

Configuration	Pass@1 (%)	Δ
Progressive build-up (vs. previous row)		
Baseline 1: Pure LLM	10.5 ± 1.9	—
Baseline 2: RAG	71.8 ± 2.5	+61.3 pp
ZhuLong (Full)	$\mathbf{82.1 \pm 3.0}$	**+10.3 pp**
Component ablation (vs. ZhuLong Full)		
ZhuLong w/o Sandbox	71.8 ± 2.5	-10.3 pp***
ZhuLong w/o Self-Expl.	$81.4 \pm 4.1^{*}$	-0.7 pp^{ns}
ZhuLong w/o Retrieval	81.0 ± 4.5	-1.1 pp^{ns}

* provisional: r1 of this arm was inadvertently run with V3 (self-exploration ON) docs; r2–r5 (clean V2) give $79.8 \pm 2.1\%$ (n=4). A clean 5-run re-measurement is pending. Significance from McNemar test on pooled per-task pass/fail across 5 runs (790 paired observations per arm) and paired bootstrap (10 000 resamples, 95% percentile CI): $^{***}p<0.001$, $^{**}p<0.01$, $^{*}p<0.05$, ns = not significant. w/o Sandbox: McNemar χ^2=36.7, $p=1.4\times10^{-9}$, bootstrap CI $[-14.2, -6.3]$ pp. w/o Retrieval: p=0.61, CI $[-4.2, +2.7]$ pp. w/o Self-Expl.: p=0.79, CI $[-3.2, +2.2]$ pp (not significant; clean n=4 gives p=0.26).

Table 5: Observation fidelity along the documentation axis (LLM, effector and loop fixed)

Rung	Configuration	Pass@1 (%)
(N) No observation	Baseline 1 (Pure LLM)	10.5 ± 1.9
(L) Low fidelity	name-only index	82.8 ± 1.0
	+ description, BM25,	
(H) High fidelity	abbreviations, synonyms	71.8 ± 2.5

(sandbox and read-back), whereas (H) is the RAG baseline whose retrieval is high-fidelity but whose effector is absent. The comparison therefore isolates the effector's dominance over documentation fidelity: even a coarse index, once the agent can execute and observe results, outperforms a rich index used only for static retrieval.

Read-back axis. The second axis is how much of the live state execution returns. Section 4.2 predicts that accuracy tracks this fidelity with the LLM held fixed; we therefore compare three closed-loop variants that differ only here: (N) *no observation*—the agent predicts from the prior alone; (B) *binary feedback*—`run_code` reports only pass/fail; and (F) *full observation*—stdout, exception traces, per-assertion outcomes, and queried state are returned (the default). The framework predicts a monotone gain from (N) to (F); the measured ordering, however, is B < N < F, a result we analyze below. We emphasize that the three rungs compress the *same* executed state to different bandwidths: (F) returns the full `error_log`/`print_log`/`result`; (B) collapses this to a single bit—`PASS` means only that the sandbox raised *no exception* (an empty `error_log`), not that the task's assertions held; and (N′) returns only the literal token `executed`. A binary `PASS` is therefore a content-free confirmation that can endorse a wrong hypothesis, which is the mechanism the analysis below appeals to.

Table 6 reports Pass@1 and the mean number of execution read-backs per variant.

Table 6: Observation-fidelity ablation along the read-back axis (RQ: Principle 2)

Observation channel (Ω)	Pass@1 (%)	Mean read-backs
(N′) None (prior only)	73.3 ± 2.2	—‡
(B) Binary pass/fail	78.1 ± 1.3	27.6‡
(F) Full observation	82.1 ± 3.0	12.1

The results recover the expected monotone ordering along the read-back axis: (N′) no feedback (73.3±2.2%) < (B) binary pass/fail (78.1±1.3%) < (F) full observation (82.1±3.0%), so each increment of observation bandwidth lifts Pass@1, in line with Principle 2. An earlier legacy probe of the (B) arm had suggested the opposite, an N′-over-B inversion (66.2±16.1%, with one anomalous run at 43.0% inflating the spread); the fresh 5-run re-measurement collapses that spread to ±1.3 and restores the predicted ordering, confirming the legacy dip was an instability artifact rather than a stable ceiling. The gaps quantify the value of observation: $N' \to B$ (+4.8 pp) is the gain from learning whether a run passes at all, and $B \to F$ (+4.0 pp) is the further gain from seeing the concrete error (type mismatch, missing state, wrong return value) that turns a categorical verdict into actionable evidence. The $N' \to F$ gap (+8.8 pp) thus measures the full practical value of *informative* observation, while the (B) arm shows that partial feedback already captures most of that value—binary pass/fail is enough to prune wrong hypotheses, but only full observation exposes the correction to apply.

Beyond accuracy, the read-back rungs differ sharply in *convergence cost*—how much execution the agent must spend to reach a submission. Table 7 aggregates per-trace execution traces retained from the runs above: the mean number of `run_code` calls per task, the wall-clock of the code-generation phase, and the count of tasks that hit the per-task time limit (a proxy for non-convergence). With the LLM and effector fixed, dropping the observation bandwidth from (F) to (B) roughly *doubles* the work the agent does per task: mean read-backs rise from 12.1 to 27.6 (∼2.3×), the code-generation wall-clock rises from 2.22 h to 4.25 h (∼1.9×), and the number of timed-out tasks rises from a mean of 3.4 to 13 (∼3.8×). The mechanism is visible in the traces: under (B) every execution returns {`"response": "PASS"/"FAIL", "error_log": "", "print_log": ""`}—the agent learns only that an exception was or was not raised, never which assertion failed or what the stack trace said, so it must hypothesize a correction blind and re-execute. One (B) trace issues 287 `run_code` calls on a single task, cycling through `PASS/FAIL` verdicts with empty error logs. This is the cost-side counterpart of the accuracy result: poorer observation does not merely cap accuracy, it inflates the search effort needed to converge, and when the budget is fixed (per-task timeout) some tasks simply fail to converge in time—exactly the "fidelity down → convergence cost up" trade-off Principle 2 implies.

We note that the fresh 5-run binary re-measurement reaches 78.1±1.3% Pass@1 (replacing the legacy single-run probe's 66.2±16.1% whose 43.0% outlier had inflated the spread), and it restores the monotone $N' < B < F$ ordering predicted by Principle 2. The convergence-cost figures reported here are aggregated from the retained execution traces of the fresh binary runs. (N′) cost figures are not recoverable: those legacy runs predate a storage migration that discarded the traces, and their wall-clock was further confounded by an infrastructure fault (license exhaustion and port failures) contemporaneous with the runs; (N′) thus reports Pass@1 only.

Table 7: Convergence cost along the read-back axis (LLM and effector fixed). "Mean `run_code`" is the per-trace execution count; "wall" is the code-generation wall-clock per run; "timeout" is the number of tasks hitting the per-task time limit. Cost rises monotonically as observation bandwidth narrows from (F) to (B).

Read-back	Pass@1 (%)	Mean `run_code`	Wall (h)	Timeout
(F) Full	82.1 ± 3.0	12.1	2.22	3.4*
(B) Binary	75.9‡	27.6	4.25	13
(N′) None	73.3†	—‡	—‡	—‡

* mean of 5 full-configuration runs (range 1–6). † provisional, n=1 probe (5-run pending). ‡ fresh n=1 from retained traces; (N′) traces unrecoverable (see text).

6.2.2 S2: The closed loop (RQ2). Principle 3 requires two things: that actions reach the design through a causal interface, and that the belief update Φ run continuously.

Effector: does the action reach the live design? Removing the sandbox from the full system drops Pass@1 from 82.1% to 71.8% (−10.3 pp; McNemar p=1.4×10^{-9}, paired bootstrap 95% CI [−14.2, −6.3] pp); adding it to the RAG baseline raises it from 71.8% to 82.1% (+10.3 pp, same significance). Without an effector no a_t is executed against the design, so b_t receives no evidence about the live state at all and the agent is reduced to a prior-only predictor. We attribute the gain to two mechanisms: execution returns concrete error messages that guide corrections, transforming one-shot generation into iterative search, and it lets the agent probe parameter combinations that documentation omits [10, 12].

Belief update: how much evidence may Φ accumulate? ZhuLong is an ICL-type autonomous agent: once the task is issued it plans, queries the API knowledge base, and executes and debugs code on its own, and it is the agent—not the harness—that decides when the result is good enough to submit. We therefore do not shorten the loop or force an early submission; we bound how often the loop *may execute*, which is what determines how much state evidence b_t ever receives. Operationally the bound is the number of `run_code` calls a trace is allowed: once the budget is spent, the call is denied at the tool boundary by a second, co-resident `PreToolUse`

hook that counts `run_code` calls per trace; its error carries a unique `[PHI-BUDGET-EXHAUSTED]` prefix because Cline's hook engine OR-merges cancellations and concatenates their reasons, which would otherwise make the anti-leak hook's denials indistinguishable from the budget hook's, and the agent keeps running under its own control until it chooses to submit. Because termination stays with the agent, we report the harness's own finish reason next to accuracy—*completed* (the agent decided it was done) versus *max iterations* or budget-denied—so that a tight budget cannot be mistaken for early convergence. In an agent with no explicit posterior, Φ is realized by what remains in the model's context, so Table 8 additionally includes a lagged channel realized as a single-slot (L=1) buffer in the `run_code` handler, whose read-back returns the state emitted by the previous execution—with a `reported_call_index` recorded so the one-step lag is auditable—leaving the agent permanently one action behind the design; the first read-back carries no state, and every other setting is unchanged. Both axes leave the LLM, the observation content and the effector untouched. Principle 3 predicts that accuracy rises with the execution budget but saturates once updates are fast enough for the design's own dynamics, since evidence obtained before the previous action is partly obsolete by the time the next action is chosen.

Table 8: Effect of bounding the evidence available to Φ (LLM, Ω and effector fixed). k is the number of `run_code` calls a trace may make—a budget on executions, not a count of design "rounds"—and "lagged" denotes a read-back that returns the state produced by the *previous* execution (the first carries none), so the agent never observes the outcome of its most recent action before acting again. Once the budget is spent the call is denied, yet the agent still terminates on its own; "Converged" is the share of traces whose finish reason is *completed*.

Update budget	Pass@1 (%)	Converged (%)	Mean gen-fail
Φ k=1	63.8 ± 3.2***	76.3	34.6
Φ k=3	69.1 ± 2.5***	80.4	27.6
Φ k=10	77.7 ± 2.2**	90.4	17.4
Φ lagged	73.0 ± 3.2***	97.5	9.0
Φ unbounded (default)	82.1 ± 3.0	99.1	10.4

All entries are 5-run means (Pass@1 ± sample std, n=5); the unbounded anchor (82.1±3.0%) is the full-system 5-run baseline (Table 4). *Converged* is the share of traces whose finish reason is *completed* (agent self-terminated), averaged over 5 runs. *Mean gen-fail* is the per-run count of tasks that produced no executable code (generation-stage failure), averaged over 5 runs; it rises as k shrinks, since a tighter execution budget leaves the agent unable to complete generation. The lagged arm is filled from the re-run that fixed the per-task buffer isolation (audited lag =1 on every executed `run_code` call). Significance vs. unbounded (McNemar on 790 paired observations per arm, paired bootstrap 95% CI): ***p|0.001, **p|0.01. k=10: p=0.006, CI [+1.3, +7.1] pp. k=3: $p=2\times10^{-13}$, CI [+8.7, +17.0] pp. k=1: $p=2\times10^{-22}$, CI [+14.4, +22.0] pp. lagged: $p=7\times10^{-9}$, CI [+4.9, +12.9] pp.

Reading Table 8 together with the *Mean gen-fail* column reveals the mechanism behind the monotone accuracy drop: as the execution budget k shrinks, the number of tasks that fail to produce any executable code rises (17.4 at k=10, 27.6 at k=3, 34.6 at k=1), which in turn drives Pass@1 downward. The budget is exhausted at the tool boundary, but the agent keeps running under its own control; with no further `run_code` calls permitted, it cannot finish or validate its generation, so the failure manifests at the generation stage rather than as an execution error. This $k\downarrow\rightarrow$ gen-fail$\uparrow\rightarrow$ Pass@1$\downarrow$ chain is the observable signature of Φ being bounded, and it is consistent with Principle 3: once the belief update can no longer accumulate state evidence, accuracy falls.

6.2.3 S3: The prior axis, the contrast arm of RQ1. Principle 1 separates knowledge from belief: the frozen prior should bound what a harness can achieve, without substituting for observation. Table 9 changes only $\mathcal{K}$—the LLM itself—with the harness held fixed. We adopt DeepSeek-V4-Pro as the primary backbone across all experiments in this paper (a deployment running the FP4-quantized variant served via an internal gateway), based on a practical trade-off between model capability and inference cost: its performance is sufficiently strong to support the key findings from both our main experiments and ablation studies, while its cost remains manageable for running the complete set of configurations reported in this work. We note that GLM-5.2 attains a marginally higher Pass@1 (83.3%) than the primary backbone (82.1%), but the two overlap within one standard deviation; DeepSeek-V4-Pro is retained as the main-backbone anchor for deployment-availability and cost reasons rather than because it is the single strongest model. The quantitative results in Table 9 are provisional and will be re-measured on the full EDA-Scripting-Bench; the qualitative finding that the underlying model remains a decisive factor—even with retrieval and sandbox in place—is consistent across backbones.

Table 9: Impact of LLM backbone (ZhuLong)

LLM Backbone	Pass@1 (%)	Δ
DeepSeek-V4-Pro	82.1 ± 3.0	—
GLM-5.2	83.3 ± 3.1	+1.2 pp^{ns}
DeepSeek-V4-Flash	16.7 ± 12.3	-65.4 pp***
Kimi-K2.6	77.0 ± 1.6	-5.1 pp**
Doubao-Seed-2.0-Pro	63.8 ± 1.4	-18.3 pp***

Significance vs. DeepSeek-V4-Pro (McNemar on 790 paired observations, paired bootstrap 95% CI): ***p|0.001, **p|0.01, ns = not significant. GLM-5.2: p=0.38, CI [−2.7, +5.4] pp. Kimi-K2.6: p=0.003, CI [+1.5, +8.4] pp. Doubao: $p=9\times10^{-23}$, CI [+13.3, +23.3] pp. Flash: $p<10^{-100}$, CI [+60.6, +69.8] pp.

Read against Principle 2, this spread is what the $\mathcal{K}$ axis can move, and it is the axis that domain adaptation would target. Principle 2 predicts that it does not substitute for the observation channel—which is why S1's fidelity ladder and S2's effector ablation move the result by more than any change of prior.

6.2.4 RQ3: re-measuring the structure per toolchain. The three slices of **EDA-Scripting-Bench** share one task schema, one assertion criterion and one harness (Section 5), so the

protocol of S1–S2 can be re-run slice by slice with nothing changed but the toolchain. We report the slices separately rather than pooled, and what the comparison is about is whether the *ordering* of the configurations and the size of the gaps survive, not whether the absolute rates agree—different tools have different API surfaces and different baseline difficulty. Table 10 reports an independent cross-language reproduction of the component ablation (the RQ2 build-up) on the SKILL slice (158 tasks) and the Tcl slice (133 tasks, spanning the Custom WaveView and Innovus runtimes), alongside the reproduction's own PyAether runs so that all four columns come from one self-consistent 5-run set per configuration. The absolute PyAether rates of this reproduction (e.g., full $81.4 \pm 1.5\%$) agree with the main-table anchor ($82.1 \pm 3.0\%$, Table 4) within one standard deviation; the comparison of interest here is the *within-language* ordering, not cross-table absolute agreement.

Table 10: Cross-language reproduction of the component ablation (RQ3). Pass@1 (%), 5-run mean±std, on EDA-Scripting-Bench. The PyAether column is the reproduction's own runs (self-consistent with its SKILL/Tcl columns, within 1σ of the main-table anchor); Tcl is split into the WaveView and Innovus runtimes (133 tasks total). The full system is the top configuration on every language; the prior-only baseline is the bottom on every language.

Configuration	PyAether	SKILL	Tcl-WaveView	Tcl-Innovus
Pure LLM	10.5 ± 1.9	33.2 ± 2.3	2.9 ± 1.1	20.0 ± 3.2
RAG (retrieval only)	72.2 ± 1.6	33.9 ± 10.6	87.0 ± 3.0	81.4 ± 5.8
ZhuLong w/o Retrieval	81.0 ± 4.5	39.0 ± 12.6	81.6 ± 3.4	61.8 ± 34.6
ZhuLong (Full)	**81.4 ± 1.5**	**60.3 ± 6.7**	**88.6 ± 2.1**	**85.3 ± 15.4**

The qualitative structure reproduces across all three languages: the complete system is the *top* configuration on PyAether, SKILL, Tcl-WaveView and Tcl-Innovus alike, and the prior-only baseline is the *bottom* on every language. The dominance of observation-plus-execution over the prior alone is therefore not an artifact of one toolchain—it follows from the domain structure, which is exactly what RQ3 is designed to test. The intermediate ordering is less stable: on PyAether the sandbox-only variant (w/o Retrieval, 81.0%) beats retrieval-only (RAG, 72.2%), whereas on both Tcl runtimes the reverse holds (RAG 87.0%/81.4% vs. w/o Retrieval 81.6%/61.8%) and on SKILL the two are statistically tied (33.9% vs. 39.0%), reflecting that different toolchains expose different amounts of usable documentation relative to executable feedback. What is invariant is the two extremes—the full system wins and the prior-only baseline loses on every language. The fidelity (Ω) and belief-update (Φ) axes of S1–S2 have not yet been re-measured on the SKILL and Tcl slices; Table 10 therefore covers the component build-up only, and the remaining axes are pending on those slices.

6.3 Error Analysis

To characterize failure modes, we inspected a representative trace from the full-system 5-run set—the run whose Pass@1 (81.6%, 129/158) is closest to the 5-run mean (82.1±3.0%, Table 4). The 29 failures comprise 17 generation-stage failures (no executable code produced) and 12 execution-stage failures (code ran but assertions failed). We classify the 12 execution-stage failures by the L1–L4 information hierarchy of Section 4: L1 (syntactic/API-name errors), L2 (parameter/signature errors), L3 (semantic/logic errors), and L4 (situated truthfulness violations—hidden type contracts, internal selection state, layout-dependent preconditions that no offline knowledge can resolve).

Table 11: Failure classification for the representative trace (129/158 pass, 29 fail). Execution-stage failures are classified by the L1–L4 hierarchy of Section 4. Generation-stage failures are broken down by cause.

Category	Count	Example Tasks
Execution-stage failures (12)		
L1: Syntactic / API-name	0	—
L2: Parameter / signature	0	—
L3: Semantic / logic	4	017, 052, 058, 144
L4: Situated / design-state	8	010, 063, 068, 074, 109, 142, 146, 155
Generation-stage failures (17)		
Timeout (>2500 s)	4	105, 123, 124, 129
Wrong save path	10	104, 106, 118, 119, 128, 131, 145, 152, 158
No code produced	3	013, 072, 120

The dominant failure mode is L4: 8 of 12 execution-stage failures (67%) are situated truthfulness violations that no amount of offline knowledge can prevent—the code is syntactically correct (L1), uses documented APIs with plausible arguments (L2–L3), yet fails because it violates an undocumented binding contract, depends on an internal selection state, or assumes a runtime context that does not hold. This directly supports the grounding-gap framework: the correctness criterion is an L4 property, and execution is the only channel that can observe the current design state.

6.3.1 L4: Situated truthfulness violations. The agent selects the correct API family and the generated code executes, but the result violates an undocumented contract that the C++-generated bindings enforce or the test oracle checks—hidden type expectations, internal state dependencies, or runtime-context preconditions that documentation does not reveal.

Case study: Task 010 (add_instance_to_schematic). The agent's code creates a schematic instance from a master cellview. Its own `run_code` tests report "ALL ASSERTIONS PASSED" and the instance is created successfully (`ntn_inst: db: 0x13f961a`). Yet the official harness fails with `AssertionError: target_cv must be an opened cellview object.` The agent passed a cellview handle that was valid in its own execution context but did not satisfy the "opened" precondition the test oracle enforces—a property of the live design state (L4), not of the API signature (L3).

Case study: Task 063 (get_cell_file_type). The agent calls a function that returns an internal type name such as `"maskLayout"`, while the test harness uses a different getter that produces `"cellFileType"`. The API name is correct and the call succeeds, but the returned type depends on which internal representation the binding exposes—a situated property of the design database, not a documented return value. The agent cannot distinguish these without executing against the live design and reading back the actual type.

Case study: Task 146 (classify_device_types). The agent's 133-line classifier passes all its own assertions, but the official harness raises `AttributeError: type object 'Param' has no attribute 'value'`. The internal `Param` type exposes attributes differently depending on the runtime context; the agent accessed `param.value` based on documentation that describes the logical property, but the binding object does not expose `.value` in the test's execution path. The attribute's availability is a situated property of the object's runtime state, not a documented API contract.

Other L4 instances follow analogous patterns: Task 074 attempted to create a `QStringList` via a self-invented wrapper that does not exist, because the C++ binding expects a specific type the documentation does not show how to construct from Python; Task 155 correctly implemented the `aeGetCommandPoint` call chain, but the function returns `False` in the headless sandbox rather than the `emyPointF` object the test expects—the return type depends on whether a GUI draw area is active, a runtime-context property the agent observed but could not resolve.

6.3.2 L3: Semantic and logic errors. The agent selects individually valid APIs, and the generated code executes without runtime errors, but the composition logic—parameter propagation, post-processing, or control flow—fails the test oracle. These failures are harder to detect because each call appears correct in isolation.

Case study: Task 052 (check_view_list). The specification required invoking `EmyNtlUtil.checkViewList` for netlist view validation. Instead, the agent implemented a custom validator using pure Python logic. The code printed "All assertions passed" in its own embedded tests, but the harness applied standard validation on a different configuration, where the heuristic returned an error message rather than `"VALID"`. The agent replaced the mandated library call with an approximation, violating the test assertion.

Case study: Task 058 (batch_oge_job_policies). The agent used substring matching (`"high" in queue_name`) instead of the equality check (`queue_name == "priority"`) required by the specification, producing wrong `misc` parameter values. The API call itself was correct, but the logic governing which input maps to which parameter value was semantically wrong—a specification-comprehension failure (L3), not a type or context failure (L4).

The remaining L3 instances—Task 017 (instance-counting logic returns 0 instead of 2) and Task 144 (property set to `None` where a concrete value is expected)—follow the same pattern: valid API calls composed with incorrect logic.

6.3.3 Generation-stage failures. The 17 generation-stage failures fall into three sub-categories (Table 11). *Timeout* (4 tasks): the agent exceeded the 2500 s per-task limit on complex multi-step design-creation tasks requiring 24–41 APIs; traces show ongoing reasoning at termination, indicating computational-budget constraints rather than capability limits. *Wrong save path* (10 tasks): the agent completed code generation and saved a valid `.py` file, but to the workspace root (`ws/`) instead of the expected `ws/pyAether_code/` subdirectory, so the evaluation harness could not locate it. *No code produced* (3 tasks): the agent's session was cancelled or ended without writing a code file, typically due to early termination on tasks with sparse documentation.

The wrong-save-path failures are notable because the generated code is likely correct—the agent solved the task but lost the point on a procedural error. This is an L1-level failure (instruction following) rather than an L4 failure (situated truthfulness), and it is amenable to harness-level mitigation (stricter output-path enforcement) rather than to improved API knowledge.

6.3.4 Implications. The L1–L4 classification makes the grounding-gap framework concrete: 67% of execution-stage failures are L4 (situated truthfulness violations), 33% are L3 (semantic/logic), and 0% are L1 or L2. No amount of offline knowledge—richer documentation, larger model, better retrieval—can prevent L4 failures, because the violated contracts are not visible until the agent acts on the live design and reads back the result. This is exactly why the execution loop is decisive (Section 6.2.2): the sandbox does not merely "run the code," it is the only channel through which the agent can observe the hidden type contracts, internal selection states, and runtime-context dependencies that constitute two-thirds of the execution-stage failures. The generation-stage failures, by contrast, are mostly procedural (wrong save path) or budget-constrained (timeout), and are amenable to harness-level fixes rather than to knowledge improvements. These findings motivate type-aware API documentation and case-level knowledge distillation, where resolved L4 failures are stored as reusable experiences, forming the basis of agent self-evolution.

7 Discussion

The error analysis in Section 6.3 shows that the 21 execution-stage failures mainly arise from API misuse and incorrect API composition. These failures are largely caused by undocumented API contracts, missing usage-level knowledge, and insufficient validation feedback, motivating type-aware retrieval and more informative execution feedback.

Beyond these specific improvements, our results carry broader implications. The dominance of sandbox execution (+10.3 pp) over static knowledge suggests that for domains with long-tailed, under-documented APIs, execution grounding is not merely beneficial but essential. Our theoretical characterization (Section 4) elevates this from an empirical observation to a structural principle: the grounding gap is bounded by observation fidelity, so execution channels—not

larger priors—are the binding constraint in such domains. This paradigm may generalize to other domain-specific scripting tasks with similar characteristics.

Limitations

Several limitations warrant acknowledgment. First, performance varies significantly across base LLMs (16.7%–83.3%), indicating that the underlying model remains a decisive factor. Second, the interactive bridge is single-threaded, limiting concurrent deployment. Third, while realistic, the main results are measured on the 158-task PyAether slice; the SKILL (158) and Tcl (133) slices are complete and a cross-language reproduction of the component ablation is reported (Section 6.2.4), but the fidelity (Ω) and belief-update (Φ) axes have not yet been re-measured on those slices, so every figure outside the RQ3 table reflects the PyAether slice. Fourth, although each configuration was evaluated over five independent runs, the benchmark size limits statistical power to certify small effects, and we do not report paired significance tests (e.g. McNemar or paired bootstrap); small gaps such as the full vs. w/o-retrieval difference (-1.1 pp) or GLM-5.2 vs. the primary backbone ($+1.2$ pp) should therefore be read as directional rather than certified. Fifth, the primary backbone is a quantized deployment (DeepSeek-V4-Pro, FP4), and we do not separately ablate quantization level, so the effect of quantization on scripting accuracy is left to future work.

8 Conclusion and Future Work

This paper presented **ZhuLong**, an execution-grounded LLM coding agent for PyAether, SKILL, and Tcl that closes the loop from generation to execution to iterative refinement via sandbox execution feedback. We further provided a **theoretical grounding**: EDA correctness is a situated (L4) property, and a Bayesian analysis shows the resulting grounding gap is bounded by observation fidelity—not model scale—explaining why runtime observation is necessary. Evaluated on **EDA-Eval-PyAether**—the first benchmark for PyAether scripting, comprising 158 real-world tasks—the complete system achieves 82.1% Pass@1 ($\pm$3.0, 5-run mean), substantially outperforming static RAG (71.8%). Ablation studies show that execution feedback and API retrieval are jointly necessary. These results demonstrate that, for domain-specific scripting with long-tail, under-documented APIs, execution grounding is not merely beneficial but essential.

Beyond the specific results, this work establishes three foundations for future research: (1) a **theoretical characterization**—the grounding gap—that diagnoses why situated (L4) correctness demands runtime observation and bounds the gap by observation fidelity; (2) a **methodological paradigm**—execution-grounded agents—that can generalize to other domains with similar characteristics (long-tail APIs, incomplete documentation, available execution sandboxes); and (3) an **evaluation infrastructure**—**EDA-Scripting-Bench** (PyAether + SKILL + Tcl, 449 tasks), whose PyAether slice EDA-Eval-PyAether anchors the main results and whose SKILL and Tcl slices support a cross-language reproduction (Section 6.2.4)—that enables systematic cross-language comparison and progress tracking for LLM-based EDA scripting. The harness, task schema and evaluation protocol will be released; the task cases themselves derive from vendor documentation and customer designs and cannot be redistributed.

Future work proceeds along three directions. First, completing the re-measurement of all ablation axes on the full EDA-Scripting-Bench (PyAether + SKILL + Tcl, 449 tasks), together with extending the benchmark to systematically cover GUI-bridged tasks on open design sessions. Second, evolving the agent through self-improvement—learning from both successful and failed execution traces to refine its retrieval, code generation, and exploration strategies. Third, adapting the architecture to additional EDA platforms, including Synopsys tools with Tcl scripting, to broaden the scope of execution-grounded EDA agents.

References

[1] Anthropic. 2024. Model Context Protocol (MCP). https://modelcontextprotocol.io/.

[2] Jacob Austin, Augustus Odena, Maxwell Nye, Maarten Bosma, Henryk Michalewski, David Dohan, Ellen Jiang, Carrie Cai, Michael Terry, Quoc Le, et al. 2021. Program Synthesis with Large Language Models. *arXiv preprint arXiv:2108.07732* (2021).

[3] Jason Blocklove, Siddharth Garg, Ramesh Karri, and Hammond Pearce. 2024. Can EDA Tool Feedback Improve Verilog Generation by LLMs? *arXiv preprint arXiv:2411.11856* (2024).

[4] Jianlv Chen, Shitao Xiao, Peitian Zhang, Kun Luo, Defu Lian, and Zheng Liu. 2024. M3-Embedding: Multi-Linguality, Multi-Functionality, Multi-Granularity Text Embeddings Through Self-Knowledge Distillation. arXiv:2402.03216 [cs.CL] https://arxiv.org/abs/2402.03216

[5] Lei Chen, Yiqi Chen, Zhufei Chu, Wenji Fang, Tsung-Yi Ho, Yu Huang, Sadaf Khan, Min Li, Xingquan Li, Yun Liang, Yibo Lin, Jinwei Liu, Yi Liu, Guojie Luo, Zhengyuan Shi, Guangyu Sun, Dimitrios Tsaras, Runsheng Wang, Ziyi Wang, Xinming Wei, Zhiyao Xie, Qiang Xu, Chenhao Xue, Evangeline F Y Young, Bei Yu, Minjie Yuan, Haoyi Zhang, Zuodong Zhang, Yuxiang Zhao, Hui-Ling Zhen, Ziyang Zheng, Binwu Zhu, Keren Zhu, and Sunan Zou. 2024. The Dawn of AI-Native EDA: Promises and Challenges of Large Circuit Models. *arXiv preprint arXiv:2403.07257* (2024). https://arxiv.org/abs/2403.07257

[6] Mark Chen, Jerry Tworek, Heewoo Jun, Qiming Yuan, Henrique Ponde de Oliveira Pinto, Jared Kaplan, Harri Edwards, Yuri Burda, Nicholas Joseph, Greg Brockman, et al. 2021. Evaluating Large Language Models Trained on Code. *arXiv preprint arXiv:2107.03374* (2021).

[7] Cline Contributors. 2024. Cline: Autonomous AI Coding Assistant. https://github.com/cline/cline.

[8] Matthijs Douze, Alexandr Guzhva, Chengqi Deng, Jeff Johnson, Gergely Szilvasy, Pierre-Emmanuel Mazaré, Maria Lomeli, Lucas Hosseini, and Hervé Jégou. 2024. The Faiss library. (2024). arXiv:2401.08281 [cs.LG]

[9] Empyrean Technology. 2026. Empyrean PyAether: Full Custom Design Platform Ecosystem. https://empyrean.com.cn/solutions/index-128.html. Accessed: 2026-06-28.

[10] Xue Jiang, Yihong Dong, Zhi Jin, and Ge Li. 2024. SEED: Customize Large Language Models with Sample-Efficient Adaptation for Code Generation. *arXiv preprint arXiv:2403.00046* (2024).

[11] Leslie Pack Kaelbling, Michael L. Littman, and Anthony R. Cassandra. 1998. Planning and acting in partially observable stochastic domains. *Artificial Intelligence* 101, 1-2 (1998), 99–134.

[12] Rajesh Kumar et al. 2026. AgentForge: Execution-Grounded Multi-Agent LLM Framework for Autonomous Software Engineering. *arXiv preprint arXiv:2604.13120* (2026). https://arxiv.org/abs/2604.13120

[13] Qiufeng Li, Rongqian Chen, Quan Cheng, Chengxuan Wang, Sizhe Tang, Wuxi Li, Duo Ding, Chia-Tung Ho, Haoxing Ren, David Z.

Pan, Tian Lan, and Weidong Cao. 2026. PDAGENT-BENCH: Characterizing, Grounding, and Architecting LLM Agents for VLSI Physical Design. *arXiv preprint arXiv:2606.17253* (2026).

[14] Bingyang Liu, Haoyi Zhang, Xiaohan Gao, Zichen Kong, Xiyuan Tang, Yibo Lin, Runsheng Wang, and Ru Huang. 2025. LayoutCopilot: An LLM-Powered Multiagent Collaborative Framework for Interactive Analog Layout Design. *IEEE Transactions on Computer-Aided Design of Integrated Circuits and Systems (TCAD)* 44, 8 (2025), 3126–3139. doi:10.1109/TCAD.2025.3529805

[15] Shang Liu, Wenji Fang, Yao Lu, Qijun Zhang, Hongce Zhang, and Zhiyao Xie. 2024. RTLCoder: Outperforming GPT-3.5 in Design RTL Generation with Our Open-Source Dataset and Lightweight Solution. In *2024 IEEE/ACM International Conference on Computer Aided Design (ICCAD)*. IEEE, New York, NY, USA. Also available at arXiv:2312.08617. https://arxiv.org/abs/2312.08617

[16] Yiyi Lu, Hoi Ian Au, Junyao Zhang, Jingyu Pan, Yiting Wang, Ang Li, Jianyi Zhang, and Yiran Chen. 2025. AutoEDA: Enabling EDA Flow Automation through Microservice-Based LLM Agents. *arXiv preprint arXiv:2508.01012* (2025). Uses MCP protocol for EDA automation. https://arxiv.org/abs/2508.01012

[17] OpenAI. 2023. *GPT-4 Technical Report*. Technical Report arXiv:2303.08774. OpenAI.

[18] Yuan Pu, Zhuolun He, Tairu Qiu, Haoyuan Wu, and Bei Yu. 2024. Customized Retrieval Augmented Generation and Benchmarking for EDA Tool Documentation QA. In *Proceedings of the IEEE/ACM International Conference on Computer-Aided Design (ICCAD)*. IEEE, New York, NY, USA. arXiv:2407.15353 [cs.CL]

[19] Yibo Rui, Yuanhang Li, Rui Wang, Ruiqi Chen, Yanxiang Zhu, Zhixiong Di, Xi Wang, and Ming Ling. 2025. ChaTCL: LLM-Based Multi-Agent RAG Framework for TCL Script Generation. In *Proceedings of the International Symposium of Electronics Design Automation (ISEDA)*. IEEE, New York, NY, USA, 736–742.

[20] Cadence Design Systems. 2024. *Virtuoso Studio SKILL Language Reference*. https://www.cadence.com

[21] Shailja Thakur, Baleegh Ahmad, Hammond A Pearce, Benjamin Tan, Brendan Dolan-Gavitt, Ramesh Karri, and Siddharth Garg. 2023. VeriGen: A Large Language Model for Verilog Code Generation. *ACM Transactions on Design Automation of Electronic Systems* 29, 3 (2023), 1–31.

[22] Sebastian Thrun, Wolfram Burgard, and Dieter Fox. 2005. *Probabilistic Robotics*. MIT Press.

[23] Ashish Vaswani, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N Gomez, Lukasz Kaiser, and Illia Polosukhin. 2017. Attention is All You Need. *Advances in Neural Information Processing Systems* 30 (2017), 5998–6008.

[24] Ning Xu, Zhaoyang Zhang, Senlin Shu, Qi Lei, Jiaqi Lv, Wensuo Wang, Tianhao Zhao, Chao Zhang, Zhaoliang Yang, Xiangyu Li, Zhaorui Su, Jingshan Li, and Xin Geng. 2026. iScript: A Domain-Adapted Large Language Model and Benchmark for Physical Design Tcl Script Generation. *arXiv preprint arXiv:2603.04476* (2026).

[25] An Yang et al. 2025. Qwen3 Technical Report. arXiv:2505.09388 [cs.CL] https://arxiv.org/abs/2505.09388